

Anexo NS-A Carga de Datos de la simulación en el guion Python

Para modificar los parámetros de la simulación el guion Python `main_NS2D.py` tiene una zona donde se concentran las su carga. A continuación se presenta dicha zona.

```
# =====
# Parametros de la simulacion y salida
# =====
# --- CONFIGURACION DE EXPORTACION ---
    flg_dot = True # True: (.) para Matlab/Octave / False: (,) para Excel ES

    # Empaquetamos elegantemente
    csv_config = {
        'sep': ',' if flg_dot else ';',
        'dec': '.' if flg_dot else ',',
    }

    flg_mode = MODO_LAMINAR # MODO_TURBULENTO<-- Elegir aqui

    PHYSICAL_PARAMS = {
        'V_in': 1.0,
        'L_y': 1.0,
        'L_x': 40.0,
        'Re': 2000.0,
        'Pr': 1,
        'nx': 201,
        'ny': 51,
        'dt': 5e-3,
        't_final': 65.0
    }
    x_targets = [0.1, 0.5, 1.0, 5.0, 10.0, 16.0]
```

El primer parámetro a configurar es el separador decimal y el separador de columnas. La configuración por omisión es usar “.” como separador decimal y “;” como separador de columnas, pero si Uds tiene configurado su sistema operativo de otra forma, para cargar los datos a excel, deberán modificar v `flg_dot`. Si usan Matlab/Octave o Python, no hará falta.

El parámetro físico a setear en primer lugar es `flg_mode` que fija si la corrida será para un flujo laminar o turbulento.

Importante El guion no hace física, si Uds corren el modo turbulento con un Re que no corresponde el guion lo hará sin ningún tipo de aviso.

La estructura `PHYSICAL_PARAMS` centraliza las propiedades geométricas L_x longitud del canal; L_y ancho del canal; los números adimensionales Re y Pr , la velocidad de entrada del flujo V_{in} la densidad del mallado n_x número de nodos en la dirección axial n_y numero de nodos en la dirección transversal. dt el paso temporal t_f durante cuanto tiempo se corre la simulación. Para Re bajos puede ser que se deba agrandar.

Anexo NS-B Estructura de los Archivos de Salida (.csv)

Los resultados de la simulación se exportan en dos archivos principales. El sufijo numérico final corresponde al identificador de la corrida.

1. Archivo Axial (axial_MODEL0...csv)

Este archivo contiene la evolución de las variables a lo largo de la longitud del canal (x). Es la herramienta principal para estudiar el desarrollo del flujo: longitud de entrada, capas límites térmica e hidrodinámica.

- **x**: Coordenada longitudinal (axial) [m].
- **u_central**: Componente axial de la velocidad en el eje central ($y = L_y/2$).
- **du_central_dx**: $\frac{\partial u}{\partial x}|_{central}$. Gradiente axial de la componente axial de la velocidad en el eje central.
- **du_dy_wall**: $\frac{\partial u}{\partial y}|_{wall}$. Gradiente transversal de la componente axial de la velocidad sobre la pared sólida.
- **T_central**: Temperatura adimensional sobre el eje central.
- **dT_central_dx**: $\frac{\partial T}{\partial x}|_{central}$ Gradiente axial de la temperatura adimensional en el eje central.
- **dT_dy_wall**: $\frac{\partial T}{\partial y}|_{wall}$. Gradiente transversal de temperatura en la pared.
- **p_central**: Presión sobre el eje central.
- **T_bulk**: Temperatura de mezcla.
- **dT_bulk_dx**: $\frac{dT_m}{dx}$ derivada axial de la temperatura de mezcla adimensional.
- **T_centro**: Temperatura adimensional en el eje central.

2. Archivo de Campos 2D (Campos2D_NS2D...csv)

Este archivo representa la estructura del flujo de pared a pared en una sección transversal (generalmente a la salida del canal, $x = L$).

- **x**: Coordenada axial [m] (desde 0 en la entrada hasta L_x en la superior).
- **y**: Coordenada transversal [m] (desde 0 en la pared inferior hasta L_y en la superior).
- **u**: Componente axial del campo de velocidades.
- **v**: Componente transversal del campo de velocidades.
- **p**: Campo de presiones.
- **T**: Campo de temperatura.
- **k / epsilon**: Energía cinética turbulenta y tasa de disipación.

Tabla 1: Guía rápida de uso de archivos

Si desea analizar...	Utilice el archivo...
Evolución de C_f o Nu vs. x	axial_...csv
Validación de caudal (integral)	Campos2D_...csv
Forma del perfil de velocidad $u(y)$	Campos2D_...csv
Longitud de entrada térmica	axial_...csv

Anexo NS-C Carga de los resultados de la simulación en Matlab/Octave

La carga de los perfiles axiales es inmediata, sin embargo la de los campos requiere de una manipulación de la estructura de datos. Este guión que corre tanto en Octave como en Matlab recupera los datos bidimensionales y genera con ellos matrices que luego se pueden usar ya sea para graficar o procesar.

```
clear all, close all
isOctave = (exist('OCTAVE_VERSION', 'builtin') ~= 0);
sDataFile='campo2D_NS2DCompleto_Re2000_Pr0.7_1422.csv';

if isOctave
    raw_data = csvread(sDataFile, 1, 0);
    x_col = raw_data(:, 1);
    y_col = raw_data(:, 2);
    u_col = raw_data(:, 3);
    v_col = raw_data(:, 4);
    p_col = raw_data(:, 5);
    T_col = raw_data(:, 6);
    disp('Datos cargados en Octave mediante csvread. ');
    nx = length(unique(x_col));
    ny = length(unique(y_col));

    X = reshape(x_col, [nx, ny])';
    Y = reshape(y_col, [nx, ny])';
    U = reshape(u_col, [nx, ny])';
    V = reshape(v_col, [nx, ny])';
    P = reshape(p_col, [nx, ny])';
    T = reshape(T_col, [nx, ny])';
else
    try
        T_table = readtable(sDataFile);
        X = T_table.x; Y = T_table.y; U = T_table.u;
        V = T_table.v; P = T_table.p; T = T_table.T;
        disp('Datos cargados en MATLAB mediante readtable. ');
    catch
        raw = csvread(sDataFile, 1, 0);
        X = raw(:,1); Y = raw(:,2); U = raw(:,3);
        V = raw(:,4); P = raw(:,5); T = raw(:,6);
    end
end

figure(1);
subplot(2,1,1);
contourf(X, Y, U, 20, 'LineColor', 'none');
colormap(gca, hot);
colorbar;
title('Campo de Velocidad Axial u(x,y)');
xlabel('x[m]'); ylabel('y[m]');

subplot(2,1,2);
contourf(X, Y, T, 20, 'LineStyle', 'none');
colormap(gca, jet);
colorbar;
hold on
skip = 5;
XS = X(1:skip:end, 1:skip:end);
YS = Y(1:skip:end, 1:skip:end);
```

```

US = U(1:skip:end, 1:skip:end);
VS = V(1:skip:end, 1:skip:end);
scale = 0.4;
US=US*scale;
VS=VS*scale;
h=quiver(XS, YS, US, VS, 0, 'k');
set(h, 'LineWidth', 1.2, 'MaxHeadSize', 0.05);
title('Campo de temperatura y vectores de Velocidad');
hold off

```

Los gráficos obtenidos se presentan a continuación:

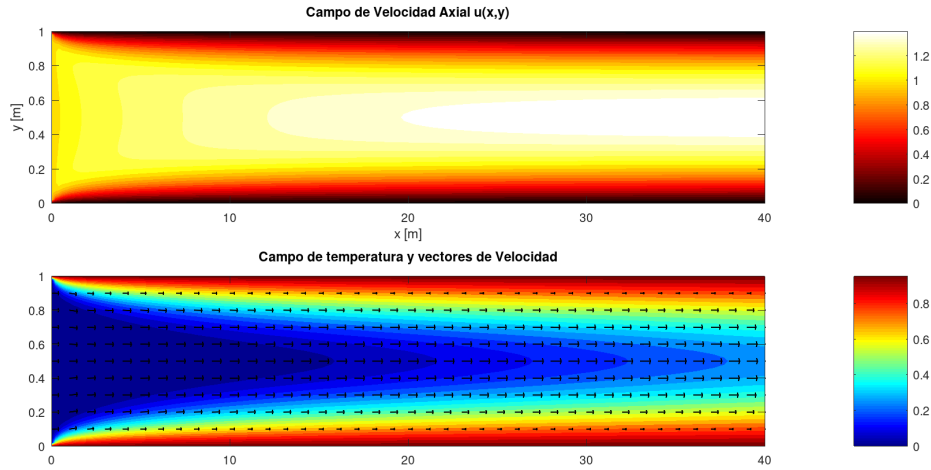


Figura 1: Distribución del campo de velocidades axial, la temperatura y los vectores velocidad normalizados. Los parámetros de la corrida son: $Re = 2000$; $Pr = 0.7$

Estructura de las matrices correspondientes a los campos 2D

En el procesamiento de los resultados se requerirá aislar la distribución de una cantidad en un x o y fijo, para ello debemos saber como se codifican las coordenadas físicas, en los elementos de la matriz (i, j) . para ello utilizaremos:

Regla de índices para la matriz de velocidad U : La matriz U tiene dimensiones (n_y, n_x) , donde cada elemento se accede como $U(\text{fila}, \text{columna})$. Para interpretar correctamente los datos: Las filas representan la coordenada transversal (y): Cada fila contiene los valores de velocidad a una altura fija desde la pared inferior ($y = 0$) hasta la superior ($y = L_y$). Las columnas representan la coordenada longitudinal (x): Cada columna contiene el perfil de velocidades completo a lo largo del canal; desde $x = 0$ en la entrada y $x = L_x$ en la salida.

En consecuencia, para extraer los datos:

- **Para obtener el perfil vertical en una posición fija de x :** Extraemos una columna completa. En Octave/MATLAB se escribe: `perfil = U(:, columna)`.
- **Para obtener la línea horizontal a lo largo del eje central (y fijo):** Extraemos una fila completa. En Octave/MATLAB se escribe: `linea = U(fila, :)`.

Anexo NS-D Regla de Escalado de la Malla según el Número de Péclet

Para unificar el efecto hidrodinámico y térmico, el refinamiento de la malla debe basarse en el número de Péclet térmico ($\mathcal{P}e = \mathcal{R}e \cdot \mathcal{P}r$).

Partiendo de una malla base de $n_y = 51$ para un número de Péclet bajo ($\mathcal{P}e = 2,000$), se propone el siguiente escalado para asegurar la resolución de ambas capas límite:

Número de Péclet ($\mathcal{P}e$)	n_y	Justificación / Comentario
$\lesssim 2,000$	51	Malla base. Difusión térmica comparable a la advección.
10,000	81	Aumento de la advección; capa límite térmica más fina.
20,000	101	Gradientes térmicos más pronunciados en la pared.
100,000	151	Transporte convectivo fuerte. Requiere mayor resolución en la región $y \rightarrow 0$.
$\gtrsim 500,000$	201	Convección dominante. Alta resolución requerida para el número de Nusselt ($\mathcal{N}u$).

Tabla 2: Escalado de nodos transversales en función del Número de Péclet.

Anexo NS-E Determinación de los parámetros de una ley de potencias

Muchas veces es posible relacionar una variable y con una variable independiente x mediante una ley de potencias del tipo:

$$y = Ax^\alpha$$

Para hallar los parámetros A y α de forma lineal, se realiza un cambio de variable aplicando logaritmo natural ($\ln$) a ambos lados de la ecuación:

$$\ln(y) = \ln(A) + \alpha \ln(x)$$

Esta expresión corresponde a la ecuación de una recta:

$$Y = mX + b$$

Donde las nuevas variables y parámetros son: $Y = \ln(y)$, $X = \ln(x)$, la pendiente $m = \alpha$, y la ordenada al origen $b = \ln(A)$.

El guion que nos permite estimar A y α para una variable independiente es el siguiente:

```
% =====  
% CASO 1: Una variable independiente (Ej: Longitud de entrada vs Reynolds)  
% =====  
X1 = [100, 500, 1000];      % Variable independiente (Ej: Re)  
Y = [5.2, 12.4, 25.1];      % Variable dependiente (Ej: Lt)  
  
log_X1 = log(X1(:));  
log_Y = log(Y(:));  
  
% Construccion de la matriz de diseno (Columna de unos + Columna de log(X1))  
X = [ones(length(log_Y), 1), log_X1];  
  
% Resolucion del sistema lineal por minimos cuadrados  
beta = X \ log_Y;  
  
beta_0 = beta(1);  
beta_1 = beta(2);  
  
% Antitransformacion para obtener los parametros de la ley de potencias  
A = exp(beta_0);  
alpha = beta_1;  
  
fprintf('==_Resultados_de_la_Regresion_Lineal_(1_Var)_==\n');  
fprintf('A_==_%.4f\n', A);  
fprintf('alpha_==_%.4f\n\n', alpha);
```

Si la ley de potencias depende de dos variables independientes x e y (como el número de Nusselt, que depende tanto de Re como de Pr), la correlación toma la forma:

$$z = Ax^\alpha y^\kappa$$

Tomando logaritmo natural, la ecuación se transforma en un plano lineal multi-variable:

$$\ln(z) = \ln(A) + \alpha \ln(x) + \kappa \ln(y)$$

El siguiente guion nos dará el valor de los parámetros α y κ . Sin embargo tenemos que ser cuidadosos en la carga de datos.

Ejemplo de aplicación: Supongan que simularon 3 números de Reynolds ($Re = \{100; 500; 1000\}$) combinados con 2 números de Prandtl ($Pr = \{0.7; 7.0\}$). Esto da un total de $3 \times 2 = 6$ corridas independientes en el simulador, de las cuales obtuvieron 6 valores distintos de Nusselt (Nu_1 a Nu_6).

Los vectores para el guion de MATLAB/Octave se deben estructurar “aplanando” esas combinaciones fila por fila de la siguiente manera:

```
% =====
% CASO 2: Dos variables independientes (Ej: Nu vs Re y Pr)
% =====
% Corrida:      1      2      3      4      5      6      -> (6 simulaciones)
v_x = [ 100;   100;   500;   500;  1000;  1000 ]; % Vector Re (x)
v_y = [ 0.7;   7.0;   0.7;   7.0;   0.7;   7.0 ]; % Vector Pr (y)
v_z = [ 5.2;  15.8;  12.4;  38.2;  25.1;  76.4 ]; % Vector Nu (z) - Resultados

log_x = log(v_x(:));
log_y = log(v_y(:));
log_z = log(v_z(:));

% Matriz de diseno con tres componentes: Unos, log(x) y log(y)
X = [ones(length(log_z), 1), log_x, log_y];

% Resolucion por minimos cuadrados multivariable
beta = X \ log_z;

% Extraccion y antitransformacion de coeficientes
A      = exp(beta(1));
alpha  = beta(2);
kappa  = beta(3);

fprintf('==_Resultados_de_la_Regresion_Multivariable_(2_Var)_==\n');
fprintf('A_=_%.4f\n', A);
fprintf('alpha_=_%.4f_(exponente_de_Re)\n', alpha);
fprintf('kappa_=_%.4f_(exponente_de_Pr)\n', kappa);

% =====
% CASO 2: Dos variables independientes (Ej: Nu vs Re y Pr)
% =====
% Cada fila representa una corrida numerica indexada de forma plana:
%      Corrida:      1      2      3      4      5      6
v_x = [ 100;   500;  1000;   100;   500;  1000 ]; % Valores de Re
v_y = [ 0.7;   0.7;   0.7;   7.0;   7.0;   7.0 ]; % Valores de Pr
v_z = [ 5.2;  12.4;  25.1;  15.8;  38.2;  76.4 ]; % Valores medidos de Nu

log_x = log(v_x(:));
log_y = log(v_y(:));
log_z = log(v_z(:));

% Matriz de diseno con tres componentes: Unos, log(x) y log(y)
X = [ones(length(log_z), 1), log_x, log_y];

% Resolucion por minimos cuadrados multivariable
beta = X \ log_z;
```



```

% Extraccion y antitransformacion de coeficientes
A      = exp(beta(1));
alpha = beta(2);
kappa = beta(3);

fprintf('==_Resultados_de_la_Regresion_Multivariable_(2_Var)_==\n');
fprintf('A_=_%.4f\n', A);
fprintf('alpha_=_%.4f_(exponente_de_x)\n', alpha);
fprintf('kappa_=_%.4f_(exponente_de_y)\n', kappa);

```